Email Generation Assistant — Evaluation Report

An LLM-powered assistant that turns an **intent**, **key facts**, and a **tone** into a polished, ready-to-send email, plus a custom evaluation harness that scores its output on three metrics and compares two models.

- **Models compared:** `openai/gpt-5.4` vs `openai/gpt-5.4-mini` (same prompt)
- **Judge model:** `anthropic/claude-sonnet-4.6`
- **Prompting technique:** Role-play + Few-shot + Chain-of-thought (delimited output)

---

1. Prompt template

Advanced prompting technique

Three techniques are combined:

1. **Role-playing** — the system prompt casts the model as an expert executive-communications writer, raising register and structure.
2. **Few-shot examples** — two worked `(intent, key_facts, tone) -> email` exemplars anchor the output format and quality bar.
3. **Chain-of-thought with delimited output** — the model plans privately in a `<thinking>` block (reader, goal, fact-weaving, what to cut) and then writes the email in an `<email>` block. The plan improves fact coverage and tone control; the delimiter lets the harness parse the email deterministically and discard the reasoning.

System prompt

```
You are an expert professional communications writer who ghost-writes email for busy
executives. You are known for email that is clear, appropriately concise, and perfectly
calibrated in tone. You never invent facts, you weave every given detail in naturally (never
as a bulleted dump), and you always include a subject line, an appropriate greeting, and a
sign-off.

You will be given an email's INTENT, a list of KEY FACTS that must all appear, and the
desired TONE.

Work in two steps:
1. Inside a <thinking> block, briefly plan: who the reader is, what outcome the email should
drive, how you will weave in each key fact, and what to leave out to keep it tight and on-
```

```
tone. Keep this under 120 words.
2. Inside an <email> block, write the final email only — starting with a "Subject:" line,
then the greeting, body, and sign-off. Put nothing else in the <email> block. Match the
requested tone precisely. Include every key fact. Do not add facts, names, dates, or numbers
that were not provided.
```

Per-scenario user message format

```
INTENT: <intent>
TONE: <tone>
KEY FACTS (every one must appear in the email):
- <fact 1>
- <fact 2>
...
```

---

2. Custom metrics

Three metrics, each normalised to `[0, 1]`. The set deliberately mixes automated techniques, as the brief requires (two LLM-as-judge, one pure-Python). The **composite** score is their mean.

Metric 1: Intent & Fact Coverage

- **Focus:** Fact Recall
- **Technique:** LLM-as-judge
- **Definition:** Does the email accomplish the stated intent and include every key fact, without inventing new ones?
- **Logic:** An impartial judge model marks each key fact present/absent (semantic match, paraphrase allowed), judges whether the intent is accomplished, and counts hallucinated claims. score = 0.70*fact_recall + 0.30*intent_accomplished - min(0.30, 0.10*hallucinated_claims), clamped to [0,1].

Metric 2: Tone Accuracy

- **Focus:** Tone Accuracy
- **Technique:** LLM-as-judge
- **Definition:** How precisely does the email's register match the requested tone?

- **Logic:** The judge rates tone match on a 1-5 rubric (5 = precise match, 1 = wrong tone). score = (rating - 1) / 4, mapping to [0,1].

Metric 3: Conciseness & Format

- **Focus:** Conciseness / Format Adherence
- **Technique:** Deterministic Python
- **Definition:** Is the email appropriately concise (vs the human reference) and well-formed (subject, greeting, sign-off, readable prose)?
- **Logic:** Pure Python, no LLM. length_score = 1 - |gen_words/ref_words - 1| (penalises bloat and under-writing). structure_score = fraction of {subject line, greeting, sign-off} present (regex). readability_score = 1 - |Flesch - 60|/60 (textstat). score = 0.5*length + 0.3*structure + 0.2*readability.

Raw evaluation data

Per-model averages

Model	Intent & Fact Coverage	Tone Accuracy	Conciseness & Format	Composite
`openai/gpt-5.4`	100.0%	77.5%	88.5%	88.7%
`openai/gpt-5.4-mini`	100.0%	80.0%	89.8%	89.9%

Model cost (OpenRouter, $ per 1M tokens)

Model	Input	Output
`openai/gpt-5.4`	$2.50	$15.00
`openai/gpt-5.4-mini`	$0.75	$4.50

Per-scenario composite scores

Scenario	`openai/gpt-5.4`	`openai/gpt-5.4-mini`
s01_followup_meeting	87.5%	89.2%
s02_rfp_details	93.5%	94.2%
s03_decline_vendor	84.8%	85.7%
s04_apology_delay	89.0%	86.2%
s05_urgent_escalation	95.8%	98.0%
s06_cold_outreach_partner	80.2%	81.7%
s07_thank_mentor	87.1%	96.6%
s08_deadline_extension	98.5%	88.7%
s09_price_increase	88.2%	89.5%
s10_webinar_invite	82.2%	89.5%

Full per-fact / per-metric detail and every generated email are in `results/results.json`, `results/results.csv`, and `results/emails.md`.

---

3. Comparative analysis

Models compared (same prompt): `openai/gpt-5.4` vs `openai/gpt-5.4-mini`
Judge: `anthropic/claude-sonnet-4.6` (independent of both, to avoid self-preference bias)

Which performed better?

`openai/gpt-5.4-mini` had the higher overall composite (89.9% vs 88.7% for `openai/gpt-5.4`) — a gap of 1.2 points. Per-metric averages:

Metric	openai/gpt-5.4-mini	openai/gpt-5.4	Gap
Intent & Fact Coverage	100.0%	100.0%	+0.0
Tone Accuracy	80.0%	77.5%	+2.5
Conciseness & Format	89.8%	88.5%	+1.3
Composite	**89.9%**	**88.7%**	**+1.2**

Biggest failure mode of the lower-performing model

`openai/gpt-5.4` 's largest deficit was on **Tone Accuracy** (77.5% vs 80.0%, a 2.5-point gap). It missed the requested register (rating ≤ 3) in 2 of 10 scenarios (s06_cold_outreach_partner, s10_webinar_invite).

Production recommendation

Recommend `openai/gpt-5.4-mini`. It is both the higher-scoring model (89.9% composite) *and* the cheaper one, so there is no quality-versus-cost trade-off to make. On cost, `openai/gpt-5.4-mini` is ~3.3x cheaper than `openai/gpt-5.4` ($4.50 vs $15.00 per 1M output tokens).